United International University

Department of Computer Science and Engineering

Structured Programming Laboratory — Fall 2026

Topic: Loop-Related Problems in C — Duration: 2.5 hours

Lab Sheet: Loop-Related Problems

Prepared by: Ashraful Islam Paran, Dept. of CSE, UIU. Follow instructions carefully and complete all assignments within the designated slot.

1. Objectives

By the end of this lab, students will be able to:

- Use `for`, `while`, and `do-while` loops to solve repetitive problems.
- Generate numerical sequences and calculate sums, averages, and products.
- Apply loop-control statements such as `break` and `continue`.
- Process the digits of an integer using arithmetic operators.
- Implement common mathematical and number-theory algorithms.
- Use nested loops and accumulated values to solve more complex problems.

2. Important Instructions

- Unless otherwise stated, assume that the inputs satisfy the conditions mentioned in each problem.
- Use loops to perform the required repetitions. Do not print a sequence by hard-coding its terms.
- Pay attention to the exact output format, including spaces, commas, labels, and line breaks.
- For problems involving a series, determine the general term before writing the loop.
- Consider boundary cases such as zero, one, equal inputs, and the smallest valid values.
- Use suitable data types to avoid unnecessary loss of precision or integer overflow.

3. Loop-Based Sequence Generation

4. Printing the Natural Number Series (*)

Write a C program that reads a positive integer N and prints the first N terms of the following series:

1 2 3 4 5 6 7 8 9 10 ...

Sample input

```
2
5
11
```

Sample output

```
1 2
1 2 3 4 5
1 2 3 4 5 6 7 8 9 10 11
```

Solution Code:

```c
#include <stdio.h>

int main() {
    int N;
    scanf("%d", &N);

    for (int i = 1; i <= N; i++) {
        printf("%d ", i);
    }
    printf("\n");
    return 0;
}
```

5. Printing the Odd Number Series (*)

Write a C program that reads a positive integer N and prints the first N terms of the following series:

$$1, 3, 5, 7, 9, 11, 13, 15, 17, \ldots$$

Sample input

```
2
5
11
```

Sample output

```
1 3
1 3 5 7 9
1 3 5 7 9 11 13 15 17 19 21
```

Solution Code:

```c
#include <stdio.h>

int main() {
    int N;
    scanf("%d", &N);

    for (int i = 1; i <= N; i++) {
        printf("%d ", 2 * i - 1);
    }
    printf("\n");
    return 0;
}
```

6. Printing an Alternating Binary Series (**)

Write a C program that reads a positive integer N and prints the first N terms of the following series:

$$1, 0, 1, 0, 1, 0, 1, 0, 1, \ldots$$

Sample input

```
1
2
3
4
7
```

Sample output

```
1
1 0
1 0 1
1 0 1 0
1 0 1 0 1 0 1
```

Solution Code:

```c
#include <stdio.h>

int main() {
    int N;
    scanf("%d", &N);

    for (int i = 1; i <= N; i++) {
        if (i % 2 != 0) {
            printf("1 ");
        } else {
            printf("0 ");
        }
    }
    printf("\n");
    return 0;
}
```

7. Average of N Numbers Without an Array (*)

Write a C program that reads an integer N, then reads N numbers and calculates their average. You must not use an array.

Sample input

```
3
10    20    30.5
2
22.4    11.1
```

Sample output

AVG of 3 inputs: 20.166667
AVG of 2 inputs: 16.750000

Solution Code:

```c
#include <stdio.h>

int main() {
    int N;
    double sum = 0.0, val;
    scanf("%d", &N);

    for (int i = 0; i < N; i++) {
        scanf("%lf", &val);
        sum += val;
    }

    printf("AVG of %d inputs: %f\n", N, sum / N);
    return 0;
}
```

8. Number Guessing Game Using `break`/`continue` (**)

Write a C program for the following scenario. Player-1 selects a number X, and Player-2 gets N attempts to guess it. For every incorrect guess, print `Wrong, N-1 Choice(s) Left!`, where the displayed number is the number of attempts remaining after that guess. If Player-2 guesses correctly, print `Right, Player-2 wins!` and terminate the guessing process immediately. If all N guesses are incorrect, print `Player-1 wins!`. Use `break` and/or `continue` in your solution. **Input format:** X, then N, followed by the N guesses.

Sample input

```
5
3
12   8   5
100
5
50   100
20
3
12   8   5
```

Sample output

```
Wrong, 2 Choice(s) Left!
Wrong, 1 Choice(s) Left!
Right, Player-2 wins!
Wrong, 4 Choice(s) Left!
Right, Player-2 wins!
Wrong, 2 Choice(s) Left!
Wrong, 1 Choice(s) Left!
```

Wrong, 0 Choice(s) Left!
Player-1 wins!

Solution Code:

```c
#include <stdio.h>

int main() {
    int X, N, guess;
    int won = 0;
    scanf("%d %d", &X, &N);

    for (int attempt = 1; attempt <= N; attempt++) {
        scanf("%d", &guess);

        if (guess == X) {
            printf("Right, Player-2 wins!\n");
            won = 1;
            break;
        }

        printf("Wrong, %d Choice(s) Left!\n", N - attempt);
    }

    if (won == 0) {
        printf("Player-1 wins!\n");
    }

    return 0;
}
```

9. Read Input Until the Character `A` Is Entered (*)

Write a C program that repeatedly reads a character from the keyboard and displays it with an input counter. Stop when the user enters the uppercase character `A`. The terminating `A` should not be displayed. **Sample input**

X
1
a
A

Sample output

Input 1: X
Input 2: 1
Input 3: a

Solution Code:

```c
#include <stdio.h>
```

```c
int main() {
    char ch;
    int count = 1;

    while (1) {
        scanf(" %c", &ch);
        if (ch == 'A') {
            break;
        }
        printf("Input %d: %c\n", count, ch);
        count++;
    }

    return 0;
}
```

10. Reverse the Digits of an Integer (**)

Write a C program that reads an integer and prints the number formed by reversing its digits.

Sample input

```
13579
4321
```

Sample output

```
97531
1234
```

Solution Code:

```c
#include <stdio.h>

int main() {
    int num, rev = 0;
    scanf("%d", &num);

    while (num > 0) {
        rev = rev * 10 + (num % 10);
        num = num / 10;
    }

    printf("%d\n", rev);
    return 0;
}
```

11. Calculate the Grade of N Students (**)

Write a C program that reads the number of students, N. For each student, read the marks obtained in attendance (out of 5), assignments (out of 10), class tests (out of 15), midterm (out of 50), and term final (out of 100). Calculate the weighted total using the following contribution:

Component	Maximum raw marks	Weight
Attendance (A)	5	5%
Assignments (HW)	10	10%
Class Tests (CT)	15	15%
Midterm (MT)	50	30%
Term Final (TF)	100	40%

Use the following grading scale based on the weighted total:

Marks	Letter Grade	Marks	Letter Grade
90–100	A	70–73	C+
86–89	A-	66–69	C
82–85	B+	62–65	C-
78–81	B	58–61	D+
74–77	B-	55–57	D
Below 55	F		

Print the grade of each student in the format `Student 1 :  A`.

Sample input

```
2
5   10   15   44.5   92.5
0   7.5   5   20   55.5
```

Sample output

```
Student 1 : A
Student 2 : F
```

Solution Code:

```c
#include <stdio.h>

int main() {
    int N;
    scanf("%d", &N);

    for (int i = 1; i <= N; i++) {
        double A, HW, CT, MT, TF;
        scanf("%lf %lf %lf %lf %lf", &A, &HW, &CT, &MT, &TF);

        double weighted = A + HW + CT + (MT / 50.0) * 30.0 + (TF / 100.0) * 40.0;

        printf("Student %d : ", i);
        if (weighted >= 90.0) printf("A\n");
        else if (weighted >= 86.0) printf("A-\n");
        else if (weighted >= 82.0) printf("B+\n");
        else if (weighted >= 78.0) printf("B\n");
        else if (weighted >= 74.0) printf("B-\n");
        else if (weighted >= 70.0) printf("C+\n");
        else if (weighted >= 66.0) printf("C\n");
        else if (weighted >= 62.0) printf("C-\n");
        else if (weighted >= 58.0) printf("D+\n");
```

```
        else if (weighted >= 55.0) printf("D\n");
        else printf("F\n");
    }

    return 0;
}
```

12. Sum of an Alternating Integer Series (**)

Write a C program that reads a positive integer N and calculates the sum of the first N terms of the following series:

$$1 - 2 + 3 - 4 + 5 - 6 + 7 - 8 + \cdots$$

Sample input

```
2
3
4
```

Sample output

```
Result: -1
Result: 2
Result: -2
```

Solution Code:

```
#include <stdio.h>

int main() {
    int N, sum = 0;
    scanf("%d", &N);

    for (int i = 1; i <= N; i++) {
        if (i % 2 != 0) {
            sum += i;
        } else {
            sum -= i;
        }
    }

    printf("Result: %d\n", sum);
    return 0;
}
```

13. Sum of the Series $1^2(2) + 2^2(3) + 3^2(4) + \cdots$ (**)

Write a C program that reads a positive integer N and calculates the sum of the first N terms of the following series:

$$1^2 \times 2 + 2^2 \times 3 + 3^2 \times 4 + 4^2 \times 5 + \cdots$$

Sample input

```
2
3
4
7
```

Sample output

```
Result: 14
Result: 50
Result: 130
Result: 924
```

Solution Code:

```c
#include <stdio.h>

int main() {
    int N;
    long long sum = 0;
    scanf("%d", &N);

    for (long long i = 1; i <= N; i++) {
        sum += (i * i) * (i + 1);
    }

    printf("Result: %lld\n", sum);
    return 0;
}
```

14. Print the Fibonacci Series (**)

Write a C program that reads a positive integer N and prints the first N terms of the Fibonacci series:

$$1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, \ldots$$

Each term after the first two is the sum of the preceding two terms. **Sample input**

```
1
2
4
7
```

Sample output

```
1
1 1
1 1 2 3
1 1 2 3 5 8 13
```

Solution Code:

```c
#include <stdio.h>
```

```c
int main() {
    int N;
    scanf("%d", &N);

    long long a = 1, b = 1;
    for (int i = 1; i <= N; i++) {
        if (i == 1 || i == 2) {
            printf("1 ");
        } else {
            long long c = a + b;
            printf("%lld ", c);
            a = b;
            b = c;
        }
    }
    printf("\n");
    return 0;
}
```

15. Print the Factorial of a Number (**)

Write a C program that reads a non-negative integer N and prints its factorial in the shown format. Display the multiplication expression as well as the final result. (For $N = 0$, use the convention $0! = 1$.) **Sample input**

```
1
2
3
4
```

Sample output

```
1! = 1 = 1
2! = 2 X 1 = 2
3! = 3 X 2 X 1 = 6
4! = 4 X 3 X 2 X 1 = 24
```

Solution Code:

```c
#include <stdio.h>

int main() {
    int N;
    scanf("%d", &N);

    printf("%d! = ", N);
    if (N == 0) {
        printf("1 = 1\n");
    } else {
        long long fact = 1;
        for (int i = N; i >= 1; i--) {
            fact *= i;
            printf("%d%s", i, (i == 1) ? "" : " X ");
```

```
        }
        printf(" = %lld\n", fact);
    }
    return 0;
}
```

16. Calculate the Combination (nCr) (**)

Write a C program that reads two integers n and r, where $n \geq r \geq 0$, and calculates the number of combinations:

$$^nC_r = \frac{n!}{r!(n-r)!}$$

Try to implement the calculation using loops. **Sample input**

```
5   2
10   3
7   7
6   1
```

Sample output

```
10
120
1
6
```

Solution Code:

```c
#include <stdio.h>

int main() {
    int n, r;
    scanf("%d %d", &n, &r);

    long long fact_n = 1, fact_r = 1, fact_n_r = 1;

    for (int i = 1; i <= n; i++) fact_n *= i;
    for (int i = 1; i <= r; i++) fact_r *= i;
    for (int i = 1; i <= (n - r); i++) fact_n_r *= i;

    long long ncr = fact_n / (fact_r * fact_n_r);
    printf("%lld\n", ncr);

    return 0;
}
```

17. Calculate x^y Using a Loop (*)

Write a C program that reads two non-negative integers x and y and calculates x^y using repeated multiplication. Do not use the `pow()` function. Assume that the result fits in the chosen data type. **Sample input**

```
5   2
2   0
6   1
0   5
```

Sample output

```
25
1
6
0
```

Solution Code:

```c
#include <stdio.h>

int main() {
    int x, y;
    long long result = 1;
    scanf("%d %d", &x, &y);

    for (int i = 0; i < y; i++) {
        result *= x;
    }

    printf("%lld\n", result);
    return 0;
}
```

18. Check Whether a Number Is Prime (**)

Write a C program that reads an integer and determines whether it is prime or not. A prime number is an integer greater than 1 that has exactly two positive divisors. **Sample input**

```
1
2
11
39
101
```

Sample output

```
Not prime
Prime
Prime
Not prime
Prime
```

Solution Code:

```c
#include <stdio.h>
```

```c
int main() {
    int n;
    scanf("%d", &n);

    if (n <= 1) {
        printf("Not prime\n");
    } else {
        int is_prime = 1;
        for (int i = 2; i * i <= n; i++) {
            if (n % i == 0) {
                is_prime = 0;
                break;
            }
        }
        if (is_prime == 1) printf("Prime\n");
        else printf("Not prime\n");
    }

    return 0;
}
```

19. Check Whether an Integer Is a Palindrome (**)

Write a C program that reads a non-negative integer and determines whether it is a palindrome number. A palindrome number remains the same when its digits are reversed. **Sample input**

```
9
91
222
12321
110
```

Sample output

```
Yes
No
Yes
Yes
No
```

Solution Code:

```c
#include <stdio.h>

int main() {
    int n, original, rev = 0;
    scanf("%d", &n);
    original = n;

    while (n > 0) {
        rev = rev * 10 + (n % 10);
        n /= 10;
```

```
    }

    if (original == rev) printf("Yes\n");
    else printf("No\n");

    return 0;
}
```

20. Sum of the Integer-Concatenation Series (**)

Write a C program that reads a positive integer N and calculates the sum of the first N terms of the following series:

$$1 + 12 + 123 + 1234 + \cdots$$

For example, when $N = 4$, calculate $1 + 12 + 123 + 1234$.

Sample input

```
1
2
3
4
```

Sample output

```
1
13
136
1370
```

Solution Code:

```
#include <stdio.h>

int main() {
    int N;
    long long total_sum = 0, current_term = 0;
    scanf("%d", &N);

    for (int i = 1; i <= N; i++) {
        current_term = current_term * 10 + i;
        total_sum += current_term;
    }

    printf("%lld\n", total_sum);
    return 0;
}
```

Keep Coding!

Consistent practice with loops builds the foundation for algorithmic thinking and problem solving.